scenario is as follows:

- We have a directory with two subdirectories, *images* and *thumb*.
- The *images* subdirectory contains many large image files; *thumb* contains thumbnails of the images, as *.jpg* files, 100x100px in image size.
- When a new image is added to the *images* directory, creation of its thumbnail in the thumb directory should be automated. If an image is modified, its thumbnail should be updated.
- The *thumbnailing* process should only be done for new or updated images, and not images that have up-to-date thumbnails.

This problem can be solved easily by creating a *Makefile* in the top-level directory, as follows:

```
FILES = $(shell  find images -type f -iname "*.jpg" | sed 's/images/
thumb/g')
CONVERT_CMD = convert -resize "100x100" $< $@
MSG = @echo "\nUpdating thumbnail" $@

all: ${FILES}
thumb/%.jpg: images/%.jpg
    $(MSG)
    $(CONVERT_CMD)
thumb/%.JPG: images/%.JPG
    $(MSG)
    $(CONVERT_CMD)
clean:
    @echo Cleaning up files..
    rm -rf thumb/*.jpg thumb/*.JPG
```

In the above *Makefile*, *FILES = $(shell find images -type f -iname "*.jpg" | sed 's/images/thumb/g')* is used to generate a list of dependency filenames. JPEG files could have the extension *.jpg* or *.JPG* (that is, differing in case). The *-iname* parameter to *find* (*find images -type f -iname "*.jpg"*) will do a case-insensitive search on the names of files, and will return files with both lower-case and upper-case extensions—for example, *images/1.jpg, images/2.jpg, images/3.JPG* and so on. The *sed* command replaces the text 'images' with 'thumb', to get the dependency file path.

When *make* is invoked, the *all* target is executed first. Since *FILES* contains a list of thumbnail files for which to check the timestamp (or if they exist), *make* jumps down to the *thumb/%.jpg* wildcard target for each thumbnail image file name. (If the extension is upper-case, that is, *thumb/3.JPG*, then *make* will look for, and find, the second wildcard target, *thumb/%.JPG*.) For each thumbnail file in the *thumb* directory, its dependency is the image file in the *images* directory. Hence, if any file (that's expected to be) in the *thumb* directory does not exist, or its timestamp is older than the dependency file in the *images* directory, the action (calling *$(CONVERT_CMD)* to create a thumbnail) is run.

Using the features we described earlier, *CONVERT_CMD* is defined before targets are specified, but it uses **recursive** assignment. Hence, the input and target filenames passed to the *convert* command are substituted from the first dependency (*$<*) and the target (*$@*) every time the action is invoked, and thus will work no matter from which action target (*thumb/%.JPG* or *thumb/%.jpg*) the action is invoked. Naturally, the 'Updating thumbnail' message is also defined using recursive assignment for the same reasons, ensuring that $(MSG) is re-evaluated every time the actions are executed, and thereby able to cope with variations in the case of the filename extension.

```
slynux@freedom:~$ make
Updating thumbnail 1.jpg
convert -resize "100x100" images/1.jpg thumb/1.jpg
… …Updating thumbnail 4.jpg
convert -resize "100x100" images/4.jpg thumb/4.jpg
```

If I edit *4.jpg* in *images* and re-run *make*, since only *4.jpg*'s timestamp has changed, a thumbnail is generated for that image:

```
slynux@freedom:~$ make
Updating thumbnail 4.jpg
convert -resize "100x100" images/4.jpg thumb/4.jpg
```

Writing a script (shell script or Python, etc) to maintain image thumbnails by monitoring timestamps would have taken many lines of code. With *make*, we can do this in just 8 lines of *Makefile*. Isn't *make* awesome?

That's all about the basics of using the *make* utility. Happy hacking till we meet again! END

By: Sarath Lakshman

The author is a hacktivist of Free and Open Source Software. He loves working on the GNU/Linux environment and contributes to the PiTiVi video editor project. He is also the developer of SLYNUX, a distro for newbies. He blogs at www.sarathlakshman.info.